Lab1:computer architectuur

Stijn Verhaeghe
Andres Bosman

February 2026

1 Introduction

Het doel van dit labo is om vertrouwd te raken met CUDA en te leren hoe we een GPU moeten programmeren. Daarom is de eerste opgave eenvoudig, het doel is om een array te flippen (figuur 1) door meerdere blocks te gebruiken met meerdere treads.

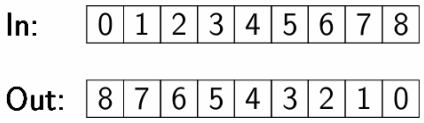


Figure 1: Array flip

2 Methode

De GPU's die we gebruiken hebben 6 sm's dus meer dan 6 blocks worden niet tegelijk uitgevoerd. Als we alle SM's tegelijk willen gebruiken dan kunnen we 6 blocks met elk 2 threads gebruiken. Vanaf de 9de thread zal een if lus ervoor zorgen dat de code niets anders doet dit gebeurd dan vanaf block 5. Het zou mogelijk zijn om dit op te lossen door eerst threadid0 te pakken van elke block en daarna pas threadid1 tot en met block 3. **van CPU naar GPU** Van de CPU naar de GPU gaan gebeurd door een global functie op te roepen en erbij te vermelden hoeveel threads en blocks je wilt. In de GPU moet je variabelen alloceren vooraleer je de functie oproept, het is dus ook belangrijk om gebruikte memory terug vrij te geven.

What is the largest possible block size (max. threads per block)?

De *CUDA C++ Programming Guide*, Release 12.3 (pagina 12), vermeldt:

> "There is a limit to the number of threads per block, since all threads of a block are expected to reside on the same streaming multiproces-

sor and must share the limited memory resources of that core. On current GPUs, a thread block may contain up to 1024 threads."

Dus je kan niet meer dan 1024 threads gebruiken per block op GPU's. (Dit kan nog veranderen in de toekomst)

3 Obeservaties / resultaten

Table 1: Vergelijking van GPU-uitvoering bij verschillende Block Sizes

Parameter	**Block size: 0**	**Block size: 1**
Input Array	`[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`	`[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`
Tijd (ms)	0.163136	0.058784
Result Array	`[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]`	`[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]`

Bij nul blocks loopt het programma zonder probleem maar er gebeurd niets in de GPU (de resultaten array blijft nul, dit is zo geinitialsieerd in de CPU). Ondanks dat er niets gebeurd heeft dit wel de langste runtime.
De tijd dat nodig is om dit te laten lopen op de CPU ontbreekt, maar we verwachten dat dit hoger gaat zijn dan op de GPU.

3.1 aantal bloks

Block size	Tijd (ms)
0	0.162528
1	0.069792
2	0.053952
3	0.050080
4	0.050656
5	0.048192
6	0.049728
7	0.048928
8	0.048928
9	0.053216
10	0.049888
11	0.048672
531	0.048384
532	0.048832
533	0.047872
534	0.049344
1000	0.184064
1001	0.061152

Table 2: Execution time versus block size

bij 1000 blocks duurt het altijd langer, we hebben dit een paar keer opnieuw laten lopen en kwamen steeds tot dezelfde conclusie. Per blok zijn er 9 threads. Maar er zijn maar 9 threads in totaal nodig. In onze code wordt er gekeken via een if/else lus of de thread ID hoger is dan de hoeveelheid elementen in de array. Deze if/else check zorgt voor een beetje overhead.

Blocks	Threads	Time (s)
1	9	0.18704
3	3	0.055744
9	1	0.061632

Table 3: Execution time for reversing an array of 9 elements

4 Conclusie

Door gebruik te maken van GPU's kan je bewerkingen die hetzelfde blijven maar over meerdere data gaan makkelijk tegelijkertijd uitvoeren. De indexes van de threads kunnen gebruikt worden als index van een analoge lus (int i die iedere keer dat een lus doorlopen wordt incrementeert maar in plaats van i gebruik je de thread-index)

5 Appendix

```cpp
#include <iostream>
#include <cuda_runtime.h>

// CUDA kernel to rev 1 arrays
__global__ void rev(int *a, int *c, int n) {
    //index wordt n-i want omgekeerd doorlopen
    // iedere element heeft zijn eigen thread in 1 blok, dus index i komt overeen met threadId
    // cores binnen 1 sm delen de ram? dus kunnen ze aan elkaars element?
    int idx = threadIdx.x  + blockIdx.x * blockDim.x;
    //threadId begint vanaf nul per blok
    //dus om index te vinden moet je het blokID (begint vanaf 0) verminigvuldigen met
    //hoeveel threads er in een blok zitten (de dimensie (1 dimensionaal voor array))
    if (idx < n) {
        c[n-idx-1] = a[idx]; //moet min 1 want index gaat van 0 tot 8
    }
}

int main() {
    //alles hier gaat over de gpu, alleen de rev wordt opgeroepen en dit is een normale code
    //er zijn hier geen branches, dit is de bedoeling, conditionele sprongen worden genomen in de rev maar dit maakt niet uit want
    // dit wrodt toch op de cpu gedaan
    const int N = 9; //van nul tot acht is array grootte maar ook aantal threads
    const int block = 1;
    int a[N];
    int c[N]; //resultaat
    int *d_a; //pointer naar begin array
    int *d_c;

    // Initialize host arrays
    std::cout << "begin array ";

    for (int i = 0; i < N; i++) {
        a[i] = i;
        c[i] = 0;
        std::cout << a[i] << " ";

    }
    std::cout << std::endl;
```

```
39
40      // Allocate device memory
41      cudaMalloc((void**)&d_a, N * sizeof(int));
42      cudaMalloc((void**)&d_c, N * sizeof(int));
43
44
45      // Copy data from host to device
46      cudaMemcpy(d_a, a, N * sizeof(int),
            cudaMemcpyHostToDevice);
47      //moet niet voor c denk ik want is toch lege array?
48      // Launch kernel (1 block of N threads)
49      rev<<<block, N>>>(d_a, d_c, N);
50
51      // Copy result back to host
52      cudaMemcpy(c, d_c, N * sizeof(int),
            cudaMemcpyDeviceToHost);
53
54      // Print results
55      std::cout << "Result:␣";
56      for (int i = 0; i < N; i++) {
57          std::cout << c[i] << "␣";
58      }
59      std::cout << std::endl;
60
61      // Free device memory
62      cudaFree(d_a);
63      cudaFree(d_c);
64
65      return 0;
66  }
```

6 ref

https://www.techpowerup.com/GPU-specs/t400-4-gb.c3843